



ABLESTACK Online Docs
ABLESTACK-V4.0-4.0.15

호스트 작업

호스트 작업

호스트 추가

게스트 VM에 더 많은 용량을 제공하기 위해 언제든지 호스트를 추가할 수 있습니다. 요구 사항 및 지침은 호스트 추가를 참조하십시오.

호스트에 대한 예약된 유지 관리 및 유지 관리 모드

호스트를 유지 관리 모드로 전환할 수 있습니다. 유지 관리 모드가 활성화되면 호스트가 새 게스트 VM을 받을 수 없게 되고 호스트에서 이미 실행 중인 게스트 VM이 유지 관리 모드가 아닌 다른 호스트로 원활하게 마이그레이션됩니다. 이 마이그레이션은 라이브 마이그레이션 기술을 사용하며 게스트 실행을 중단하지 않습니다.

vCenter 및 유지 관리 모드

vCenter 호스트에서 유지 관리 모드를 시작하려면 vCenter와 Mold를 함께 사용해야 합니다. Mold와 vCenter에는 밀접하게 함께 작동하는 별도의 유지 관리 모드가 있습니다.

1. 호스트를 Mold의 "예약된 유지 관리" 모드로 전환합니다. 이는 vCenter 유지 관리 모드를 호출하지 않고 VM이 호스트에서 마이그레이션되도록 합니다.

Mold 유지 관리 모드가 요청되면 호스트는 먼저 유지 관리 준비 상태로 이동합니다. 이 상태에서는 새 게스트 VM 시작의 대상이 될 수 없습니다. 그런 다음 모든 VM이 서버에서 마이그레이션됩니다. 라이브 마이그레이션은 호스트에서 VM을 이동하는 데 사용됩니다. 이를 통해 게스트를 중단하지 않고 게스트를 다른 호스트로 마이그레이션 할 수 있습니다. 이 마이그레이션이 완료되면 호스트는 유지 관리 준비 모드로 전환됩니다.

2. UI에 "유지 보수 준비" 표시기가 나타날 때까지 기다립니다.
3. 이제 vCenter를 사용하여 호스트를 유지하는 데 필요한 모든 작업을 수행합니다. 이 시간 동안 호스트는 새 VM 할당의 대상이 될 수 없습니다.
4. 유지 관리 작업이 완료되면 다음과 같이 호스트를 유지 관리 모드에서 해제합니다.
 - a. 먼저 vCenter를 사용하여 vCenter 유지 관리 모드를 종료합니다. 그러면 호스트가 Mold를 다시 활성화할 준비가 됩니다.
 - b. 그런 다음 Mold의 관리자 UI를 사용하여 Mold 유지 관리 모드를 취소합니다. 호스트가 다시 온라인 상태가 되면 해당 호스트에서 마이그레이션된 VM을 수동으로 다시 마이그레이션하고 새 VM을 추가할 수 있습니다.

XenServer 및 유지 관리 모드

XenServer의 경우 XenCenter의 유지 관리 모드 기능을 사용하여 일시적으로 서버를 오프라인으로 전환할 수 있습니다. 서버를 유지 관리 모드로 전환하면 실행 중인 모든 VM이 동일한 풀의 다른 호스트로 자동으로 마이그레이션됩니다. 서버가 풀 마스터인 경우 풀에 대한 새 마스터도 선택됩니다. 서버가 유지 관리 모드인 동안에는 VM을 만들거나 시작할 수 없습니다.

서버를 유지 관리 모드로 전환하려면 :

1. 리소스 창에서 서버를 선택한 후 다음 중 하나를 수행합니다.
 - 마우스 오른쪽 버튼을 클릭한 다음 바로 가기 메뉴에서 유지 관리 모드 시작을 클릭합니다.

- 서버 메뉴에서 유지 관리 모드 시작을 클릭합니다.

2. 유지 관리 모드 입력을 클릭합니다.

리소스 창 의 서버 상태는 실행 중인 모든 VM이 서버에서 성공적으로 마이그레이션된 시기를 보여줍니다.

서버를 유지 관리 모드에서 해제하려면 :

1. 리소스 창에서 서버를 선택한 후 다음 중 하나를 수행합니다.

- 마우스 오른쪽 버튼을 클릭한 다음 바로 가기 메뉴에서 유지 관리 모드 종료를 클릭합니다.
- 서버 메뉴에서 유지 관리 모드 종료를 클릭합니다.

2. 유지 관리 모드 종료를 클릭합니다.

Zone, Pod 및 클러스터 비활성화 및 활성화

클라우드에서 영구적으로 제거하지 않고 Zone, Pod 또는 클러스터를 활성화하거나 비활성화할 수 있습니다. 이는 유지 관리 또는 클라우드 인프라의 일부를 불안정하게 만드는 문제가 있을 때 유용합니다. 상태가 Enabled로 돌아갈 때까지 비활성화된 Zone, Pod 또는 클러스터에 대한 새 할당이 수행되지 않습니다. Zone, Pod 또는 클러스터가 클라우드에 처음 추가되면 기본적으로 비활성화됩니다.

Zone, Pod 또는 클러스터를 비활성화 및 활성화하려면 :

1. Mold UI에 관리자로 로그인합니다.
2. 왼쪽 탐색 모음에서 인프라스트럭처를 클릭합니다.
3. Zone을 클릭합니다.
4. 목록에서 작업할 Zone을 클릭합니다.
5. Zone을 비활성화하거나 활성화하는 경우 Zone 활성화 또는 Zone 비활성화 버튼을 클릭합니다.
6. 작업할 Pod를 클릭합니다.
7. Pod를 비활성화하거나 활성화하는 경우 Pod 활성화 또는 Pod 비활성화 버튼을 클릭합니다.
8. 작업할 클러스터를 클릭합니다.
9. 클러스터를 비활성화하거나 활성화하는 경우 클러스터 활성화 또는 클러스터 비활성화 버튼을 클릭합니다.

호스트 제거

필요에 따라 호스트를 클라우드에서 제거할 수 있습니다. 호스트를 제거하는 절차는 하이퍼바이저 유형에 따라 다릅니다.

XenServer 및 KVM 호스트 제거

노드는 유지 관리 모드가 될 때까지 클러스터에서 제거할 수 없습니다. 이렇게 하면 모든 VM이 다른 호스트로 마이그레이션되었습니다. 클라우드에서 호스트를 제거하려면 :

1. 노드를 유지 보수 모드로 설정하십시오.
"호스트에 대한 예약된 유지 관리 및 유지 관리 모드"를 참조하십시오.
2. KVM의 경우 클라우드 에이전트 서비스를 중지하십시오.

3. UI 옵션을 사용하여 노드를 제거하십시오.

그런 다음 호스트의 전원을 끄고 IP 주소를 다시 사용하고 다시 설치할 수 있습니다.

vSphere 호스트 제거

이러한 유형의 호스트를 제거하려면 먼저 "호스트에 대한 예약된 유지 관리 및 유지 관리 모드"에 설명된 대로 유지 관리 모드로 전환하십시오. 그런 다음 Mold를 사용하여 호스트를 제거합니다. Mold는 Mold를 사용하여 제거된 호스트에 명령을 전달하지 않습니다. 그러나 호스트는 vCenter 클러스터에 여전히 존재할 수 있습니다.

호스트 다시 설치

호스트를 유지 관리 모드로 전환한 다음 제거한 후 다시 설치할 수 있습니다. 호스트가 다운되어 유지 관리 모드로 전환할 수 없는 경우 다시 설치하기 전에 제거해야 합니다.

호스트에서 하이퍼바이저 유지

호스트에서 하이퍼바이저 소프트웨어를 실행할 때 하이퍼바이저 공급 업체에서 제공하는 모든 핫픽스가 적용되었는지 확인하십시오. 하이퍼바이저 공급 업체의 지원 채널을 통해 하이퍼바이저 패치 릴리스를 추적하고 패치가 릴리스 된 후 가능한 한 빨리 적용하십시오. Mold는 필수 하이퍼바이저 패치를 추적하거나 알리지 않습니다. 호스트가 제공된 하이퍼바이저 패치로 완전히 최신 상태여야 합니다. 하이퍼바이저 공급 업체는 최신 패치가 아닌 시스템을 지원하지 않을 가능성이 높습니다.

Note

최신 핫픽스가 없으면 데이터가 손상되고 VM이 손실될 수 있습니다.

(XenServer) 자세한 내용은 Mold 기술 자료에서 XenServer에 대한 권장 핫픽스를 참조하십시오.

호스트 비밀번호 변경

XenServer 노드, KVM 노드 또는 vSphere 노드의 암호는 데이터베이스에서 변경할 수 있습니다. 클러스터의 모든 노드는 동일한 암호를 가져야 합니다.

노드의 비밀번호를 변경하려면 :

1. 클러스터의 모든 호스트를 식별합니다.
2. 클러스터의 모든 호스트에서 비밀번호를 변경하십시오. 이제 호스트의 비밀번호와 Mold에 알려진 비밀번호가 일치하지 않습니다. 두 암호가 일치할 때까지 클러스터에 대한 작업이 실패합니다.
3. 데이터베이스의 암호가 암호화된 경우 데이터베이스에 추가하기 전에 데이터베이스 키를 사용하여 새 암호를 암호화해야 합니다.

```
java -classpath /usr/share/cloudstack-common/lib/jasypt-1.9.0.jar \
org.jasypt.intf.cli.JasyptPBESStringEncryptionCLI \
encrypt.sh input="newrootpassword" \
password="databasekey" \
verbose=false
```

4. 암호를 변경하는 클러스터의 호스트에 대한 호스트 ID 목록을 가져옵니다. 이러한 호스트 ID를 확인하려면 데이터베이스에 액세스해야 합니다. 암호를 변경하려는 각 호스트 이름 "h"(또는 vSphere 클러스터)에 대해 다음을 실행합니다.

```
mysql> SELECT id FROM cloud.host WHERE name like '%h%';
```

5. 단일 ID를 반환해야 합니다. 이러한 호스트에 대해 이러한 ID 세트를 기록하십시오. 이제 호스트에 대한 host_details 행 ID를 검색하십시오.

```
mysql> SELECT * FROM cloud.host_details WHERE name='password' AND host_id={previous step ID};
```

6. 데이터베이스에서 호스트의 비밀번호를 업데이트하십시오. 이 예에서는 호스트 ID가 5 및 12이고 host_details ID가 8 및 22인 호스트의 암호를 "password"로 변경합니다.

```
mysql> UPDATE cloud.host_details SET value='password' WHERE id=8 OR id=22;
```

오버 프로비저닝 및 서비스 제공 제한

(XenServer, KVM 및 VMware에 대해 지원됨) CPU 및 메모리 (RAM) 오버 프로비저닝 요소는 클러스터의 각 호스트에서 실행할 수 있는 VM 수를 변경하기 위해 각 클러스터에 대해 설정할 수 있습니다. 이는 리소스 사용을 최적화하는 데 도움이 됩니다. 초과 프로비저닝 비율을 높이면 더 많은 리소스 용량이 사용됩니다. 비율이 1로 설정되면 오버 프로비저닝이 수행되지 않습니다.

관리자는 cpu.overprovisioning.factor 및 mem.overprovisioning.factor 전역 구성 변수에서 전역 기본 오버 프로비저닝 비율을 설정할 수도 있습니다. 이러한 변수의 기본값은 1입니다. 오버 프로비저닝은 기본적으로 꺼져 있습니다.

오버 프로비저닝 비율은 Mold의 용량 계산에서 동적으로 대체됩니다. 예를 들면 :

용량 = 2GB 오버 프로비저닝 요소 = 2 오버 프로비저닝 후 용량 = 4GB

이 구성으로 각각 1GB의 VM 3 개를 배포한다고 가정합니다.

사용됨 = 3GB 사용 가능 = 1GB

관리자는 메모리 오버 프로비저닝 비율을 지정할 수 있으며 클러스터별로 CPU 및 메모리 오버 프로비저닝 비율을 모두 지정할 수 있습니다.

특정 클라우드에서 각 호스트에 대한 최적의 VM 수는 하이퍼바이저, 스토리지 및 하드웨어 구성과 같은 요소의 영향을 받습니다. 이는 동일한 클라우드의 각 클러스터마다 다를 수 있습니다. 단일 글로벌 오버 프로비저닝 설정은 클라우드의 모든 다른 클러스터에 대해 최상의 활용도를 제공할 수 없습니다. 가장 낮은 공통분모로 설정해야 합니다. 클러스터별 설정은 Mold 배치 알고리즘이 VM 배치를 결정하는 위치와 관계없이 리소스 활용도를 높이기 위해 더 세분화됩니다.

오버 프로비저닝 설정은 전용 리소스 (특정 클러스터를 계정에 할당)와 함께 사용하여 서로 다른 계정에 다양한 수준의 서비스를 효과적으로 제공할 수 있습니다. 예를 들어, 더 비싼 서비스 수준에 대한 비용을 지불하는 계정은 오버 프로비저닝 비율이 1인 전용 클러스터에 할당되고 비용이 낮은 계정은 비율이 2인 클러스터에 할당될 수 있습니다.

새 호스트가 클러스터에 추가되면 Mold는 호스트가 해당 클러스터에 대해 구성된 CPU 및 RAM 오버 프로비저닝을 수행할 수 있는 기능을 가지고 있다고 가정합니다. 호스트가 설정된 오버 프로비저닝 수준에 실제로 적합한지 확인하는 것은 관리자의 몫입니다.

XenServer 및 KVM의 오버 프로비저닝에 대한 제한 사항

- XenServer에서는이 하이퍼바이저의 제약으로 인해 4보다 큰 오버 프로비저닝 요소를 사용할 수 없습니다.
- KVM 하이퍼바이저는 VM에 대한 메모리 할당을 동적으로 관리할 수 없습니다. Mold는 VM이 사용할 수 있는 최소 및 최대 메모리량을 설정합니다. 하이퍼바이저는 메모리 경합에 따라 설정된 제한 내에서 메모리를 조정합니다.

오버 프로비저닝에 대한 요구 사항

오버 프로비저닝이 제대로 작동하려면 몇 가지 전제 조건이 필요합니다. 이 기능은 OS 유형, 하이퍼바이저 기능 및 특정 스크립트에 따라 다릅니다. 이러한 요구 사항이 충족되는지 확인하는 것은 관리자의 책임입니다.

Balloon Driver

모든 VM에는 Balloon Driver가 설치되어 있어야 합니다. 하이퍼바이저는 벌룬 드라이버와 통신하여 메모리를 확보하고 VM에서 사용할 수 있도록 합니다.

XenServer

벌룬 드라이버는 xen pv 또는 PVHVM 드라이버의 일부로 찾을 수 있습니다. xen pvhvm 드라이버는 업스트림 Linux 커널 2.6.36 이상에 포함되어 있습니다.

VMware

벌룬 드라이버는 VMware 도구의 일부로 찾을 수 있습니다. 오버 프로비저닝된 클러스터에 배포된 모든 VM에는 VMware 도구가 설치되어 있어야 합니다.

KVM

virtio 드라이버를 지원하려면 모든 VM이 필요합니다. 이러한 드라이버는 모든 Linux 커널 버전 2.6.25 이상에 설치됩니다. 관리자는 virtio 구성에서 CONFIG_VIRTIO_BALLOON=y를 설정해야 합니다.

하이퍼바이저 기능

하이퍼바이저는 메모리 벌루닝을 사용할 수 있어야 합니다.

XenServer

하이퍼바이저의 DMC (Dynamic Memory Control) 기능을 활성화해야 합니다. XenServer Advanced 이상 버전에만이 기능이 있습니다.

VMware, KVM

메모리 벌루닝은 기본적으로 지원됩니다.

오버 프로비저닝 비율 설정

루트 관리자가 CPU 및 RAM 오버 프로비저닝 비율을 설정할 수 있는 두 가지 방법이 있습니다. 첫째, 새 클러스터가 생성될 때 전역 구성 설정 `cpu.overprovisioning.factor` 및 `mem.overprovisioning.factor`가 적용됩니다. 나중에 기존 클러스터의 비율을 수정할 수 있습니다.

변경 후 배포된 VM 만 새 설정의 영향을 받습니다. 변경 전에 배포된 VM이 새로운 오버 프로비저닝 비율을 채택하도록 하려면 VM을 중지했다가 다시 시작해야 합니다. 이 작업이 완료되면 Mold는 새로운 오버 프로비저닝 비율에 따라 사용 및 예약된 용량을 다시 계산하거나 확장하여 Mold가 여유 용량의 양을 올바르게 추적하도록 합니다.

Note

사용 가능한 용량의 새 값이 새 VM을 수용할 만큼 충분히 높지 않은 경우 용량 재계산이 진행되는 동안 추가 새 VM을 배포하지 않는 것이 더 안전합니다. 새 사용/사용 가능한 값이 사용 가능해질 때까지 기다리면 원하는 모든 새 VM을 위한 공간이 있는지 확인합니다.

기존 클러스터의 오버 프로비저닝 비율을 변경하려면 :

1. Mold UI에 관리자로 로그인하십시오.
2. 왼쪽 탐색 모음에서 인프라스트럭처를 클릭합니다.
3. 작업할 클러스터를 클릭합니다.
4. 설정 탭을 클릭합니다.
5. CPU 오버 커밋 비율을 변경할 경우에는 `cpu.overprovisioning.factor` 값을, RAM 오버 커밋 비율을 변경할 경우에는 `mem.overprovisioning.factor` 값을 입력합니다.

Note

XenServer에서는 이 하이퍼바이저의 제약으로 인해 4보다 큰 오버 프로비저닝 요소를 사용할 수 없습니다.

서비스 제공 제한 및 오버 프로비저닝

코어 수에는 서비스 제공 제한 (예 : 1GHz, 1 코어)이 엄격하게 적용됩니다. 예를 들어, 하나의 코어 서비스를 제공하는 게스트는 호스트의 다른 활동과 관계없이 하나의 코어 만 사용할 수 있습니다.

기가 헤르츠에 대한 서비스 제공 제한은 CPU 리소스에 대한 경합이 있는 경우에만 적용됩니다. 예를 들어 게스트가 2GHz 코어가 있는 호스트에서 1GHz 서비스 제공으로 생성되었고 해당 게스트가 호스트에서 실행되는 유일한 게스트라고 가정합니다. 게스트는 전체 2GHz를 사용할 수 있습니다. 여러 게스트가 CPU를 사용하려고 할 때 CPU 리소스를 예약하는 데 가중치 요소가 사용됩니다. 가중치는 서비스 제공의 클럭 속도를 기반으로 합니다. 게스트는 서비스 제공의 GHz에 비례하는 CPU 할당을 받습니다. 예를 들어, 2GHz 서비스 오퍼링에서 생성된 게스트는 1GHz 서비스 오퍼링에서 생성된 게스트에 비해 두 배의 CPU 할당을 받습니다. Mold는 메모리 오버 프로비저닝을 수행하지 않습니다.

VLAN 프로비저닝

Mold는 호스트의 VLAN에 연결된 인터페이스를 자동으로 생성하고 파괴합니다. 일반적으로 관리자는 이 프로세스를 관리할 필요가 없습니다.

Mold는 하이퍼바이저 유형에 따라 VLAN을 다르게 관리합니다. XenServer 또는 KVM의 경우 VLAN은 사용될 호스트에서만 생성된 다음 이를 필요로 하는 모든 게스트가 종료되거나 다른 호스트로 이동되면 폐기됩니다.

vSphere의 경우 VLAN이 필요한 특정 호스트에서 실행 중인 게스트가 없더라도 클러스터의 모든 호스트에 VLAN이 프로비저닝됩니다. 이를 통해 관리자는 대상 호스트에 VLAN을 만들지 않고도 vCenter에서 라이브 마이그레이션 및 기타 기능을 수행할 수 있습니다. 또한 VLAN은 더 이상 필요하지 않을 때 호스트에서 제거되지 않습니다.

각 물리적 네트워크에 스위치와 같은 자체 기본 계층 2 인프라가 있는 경우 서로 다른 물리적 네트워크에서 동일한 VLAN을 사용할 수 있습니다. 예를 들어 고급 영역 설정에서 물리적 네트워크 A와 B를 배포하는 동안 VLAN 범위를 500에서 1000까지 지정할

수 있습니다. 이 기능을 사용하면 다른 물리적 NIC에 추가 레이어 2 물리적 인프라를 설정하고 VLAN이 부족한 경우 동일한 VLAN 세트를 사용할 수 있습니다. 또 다른 이점은 서로 다른 고객에 대해 동일한 IP 세트를 사용할 수 있다는 것입니다. 각 고객은 서로 다른 물리적 NIC에 있는 게스트 네트워크와 자체 라우터가 있습니다.

VLAN 할당 예

공용 및 게스트 트래픽에는 VLAN이 필요합니다. 다음은 VLAN 할당 체계의 예입니다.

VLAN IDs	Traffic type	Scope
500 미만	관리 트래픽.	관리 목적으로 예약되어 있습니다. Mold 소프트웨어는 하이퍼바이저, 시스템 VM에 액세스할 수 있습니다.
500-599	공용 트래픽을 전달하는 VLAN.	Mold 계정.
600-799	게스트 트래픽을 전달하는 VLAN.	Mold 계정. 이 풀에서 계정별 VLAN이 선택됩니다.
800-899	게스트 트래픽을 전달하는 VLAN.	Mold 계정. Mold 관리자가 해당 계정에 할당하기 위해 선택한 계정별 VLAN.
900-999	게스트 트래픽을 전달하는 VLAN	Mold 계정. 프로젝트, 도메인 또는 모든 계정으로 범위를 지정할 수 있습니다.
1000 이상	향후 사용을 위해 예약 됨	

비 연속 VLAN 범위 추가

Mold는 네트워크에 연속되지 않은 VLAN 범위를 추가할 수 있는 유연성을 제공합니다. 관리자는 Zone을 생성하는 동안 기존 VLAN 범위를 업데이트하거나 연속되지 않은 여러 VLAN 범위를 추가할 수 있습니다. UpdatephysicalNetwork API를 사용하여 VLAN 범위를 확장할 수도 있습니다.

1. 관리자 또는 최종 사용자로 Mold UI에 로그인합니다.
2. VLAN 범위가 이미 존재하지 않는지 확인하십시오.
3. 왼쪽 탐색에서 인프라스트럭처를 선택합니다.
4. Zone을 클릭합니다.
5. 목록에서 작업할 Zone을 클릭합니다.
6. 물리적 네트워크 탭을 클릭합니다.
7. 다이어그램의 게스트 노드에서 구성을 클릭합니다.

8. 물리 네트워크 업데이트를 클릭 버튼을 눌러 VLAN 범위를 편집합니다.

이제 VLAN 범위 필드를 편집할 수 있습니다.

9. '-'로 구분하여 VLAN 범위의 시작과 끝을 지정하십시오.

10. 확인 버튼을 클릭하십시오.

격리된 네트워크에 VLAN 할당

Mold는 격리된 네트워크에 대한 VLAN 할당을 제어하는 기능을 제공합니다. 루트 관리자는 공유 네트워크에 대해 수행되는 방식으로 네트워크가 생성될 때 VLAN ID를 할당할 수 있습니다.

이전 동작도 지원됩니다. VLAN은 네트워크가 구현된 상태로 전환될 때 물리적 네트워크의 VNET 범위에서 네트워크에 무작위로 할당됩니다. VLAN은 네트워크 가비지 수집의 일부로 네트워크가 종료될 때 VNET 풀로 다시 해제됩니다. VLAN은 다시 구현될 때 동일한 네트워크 또는 다른 네트워크에서 재사용할 수 있습니다. 이후에 네트워크를 구현할 때마다 새 VLAN을 할당할 수 있습니다.

일반 사용자 또는 도메인 관리자는 물리적 네트워크 토폴로지를 인식하지 못하므로 루트 관리자 만 VLAN을 할당할 수 있습니다. 네트워크에 할당된 VLAN도 볼 수 없습니다.

격리된 네트워크에 VLAN을 할당하려면

1. 다음을 지정하여 네트워크 오퍼링을 작성하십시오.

- 게스트 유형 : 격리됨을 선택합니다.
- VLAN 지정 : 옵션을 선택합니다. 자세한 내용은 Mold 설치 가이드를 참조하십시오.

2. 이 네트워크 오퍼링을 사용하여 네트워크를 작성하십시오.

3. VPC 계층 또는 격리된 네트워크를 생성할 수 있습니다.

4. 네트워크를 만들 때 VLAN을 지정합니다.

5. VLAN이 지정되면 CIDR 및 게이트웨이가 이 네트워크에 할당되고 상태가 설정으로 변경됩니다. 이 상태에서 네트워크는 가비지 수집되지 않습니다.

Note

네트워크에 할당된 VLAN은 변경할 수 없습니다. VLAN은 전체 수명주기 동안 네트워크와 함께 유지됩니다.

대역 외 관리

Mold는 루트 관리자에게 물리적 호스트에서 지원되는 대역 외 관리 인터페이스 (예 : IPMI, iLO, DRAC 등)를 구성하고 사용하여 켜기, 끄기, 재설정 등과 같은 호스트 전원 작업을 관리할 수 있는 기능을 제공합니다. 기본적으로, IPMI 2.0베이스 보드 컨트롤러는 IPMI 2.0 관리 작업을 수행하기 위해 `ipmitool` 사용하는 Mold의 `IPMITOOL` 대역 외 관리 드라이버를 통해 즉시 지원됩니다.

Mold는 대역 외 관리를 위해 Redfish 프로토콜도 지원합니다. Redfish는 서버를 제어하기 위해 HTTP REST API를 제공하며 최신 시스템에서 널리 채택되었습니다. Mold의 Redfish 대역 외 드라이버에서 지원하는 명령은 IPMITOOL 드라이버에서 지원하는 것과 동일합니다.

참고 : 지금까지 Mold는 공식적으로 Dell 및 Supermicro 시스템용 Redfish 프로토콜을 지원합니다.

다음은 이 기능의 다양한 측면을 제어하는 몇 가지 전역 설정입니다.

글로벌 설정	기본값	기술
outofbandmanagement.action.timeout	60	대역 외 관리 작업 시간 제한 (초), 클러스터별로 구성 가능
outofbandmanagement.ipmitool.interface	Lanplus	사용할 대역 외 관리 Ipmitool 드라이버 인터페이스입니다. 유효한 값은 lan, lanplus 등입니다.
outofbandmanagement.ipmitool.path	/usr/bin/ipmitool	Ipmitool 드라이버에서 사용하는 대역 외 관리 ipmitool 경로
outofbandmanagement.ipmitool.retries	1	대역 외 관리 Ipmitool 드라이버 재시도 옵션 -R

outofbandmanagement.sync.poolsize	50	대역 외 관리 백그라운드 동기화 스레드 풀 크기 50
redfish.ignore.ssl	True	기본값은 false이며 클라이언트 요청이 SSL을 사용할 때 인증서의 유효성을 검사하도록합니다. true로 설정하면 redfish 클라이언트가 Redfish 서버에 요청을 보낼 때 SSL 인증서 유효성 검사를 무시합니다.
redfish.use.https	True	모든 연결에 HTTPS / SSL을 사용합니다.

`outofbandmanagement.sync.poolsize` 설정을 변경하려면 Mold 관리 서버가 시작될 때 로드 시간 동안 스레드 풀 및 백그라운드 (전원 상태) 동기화 스레드가 구성되므로 관리 서버를 다시 시작해야 합니다. 나머지 전역 설정은 관리 서버를 다시 시작하지 않고도 변경할 수 있습니다.

`outofbandmanagement.sync.poolsize` 는 한 번에 실행할 수 있는 ipmitool 백그라운드 전원 상태 스캐너의 최대 수입입니다. 보유한 최대 호스트 수에 따라 관리 서버 호스트가 견딜 수 있는 스트레스의 정도에 따라 값을 늘리거나 줄일 수 있습니다. 단일 라운드에서 백그라운드 전원 상태 동기화 스캔을 완료하려면 라운드 * `outofbandmanagement.action.timeout` / `outofbandmanagement.sync.poolsize` 초에서 최대 총 대역 외 관리 활성화 호스트 수가 필요합니다.

이 기능을 사용하려면 루트 관리자가 먼저 UI 또는 `configureOutOfBandManagement` API를 사용하여 호스트에 대한 대역 외 관리를 구성해야 합니다. 다음으로 루트 관리자가 이를 활성화해야 합니다. 이 기능은 Zone, 클러스터 또는 호스트에서 활성화 또는 비활성화할 수 있습니다.

대역 외 관리가 호스트에 대해 구성되고 활성화되면 (Zone 또는 클러스터 수준에서 비활성화되지 않은 상태로 제공됨) 루트 관리자는 켜기, 끄기, 재설정, 주기, 소프트 및 상태와 같은 전원 관리 작업을 실행할 수 있습니다.

호스트가 유지 관리 모드에 있는 경우 루트 관리자는 여전히 전원 관리 작업을 수행할 수 있지만 UI에는 경고가 표시됩니다.

Note

IPMI 기반 대역 외 관리 및 호스트 HA는 기본 ipmitool 버전을 사용하는 Centos 8에서 작동하지 않을 수 있습니다. ipmitool-1.8.18-12.el8_1.x86_64.rpm을 설치하면 문제가 해결될 수 있습니다. IPMI 기반 대역 외 관리 및 호스트 HA 기능을 사용하기 전에 물리적 장비에서 ipmitool을 테스트해야 합니다.

보안

Mold에는 내장된 인증 기관 (CA) 프레임 워크와 자체 서명된 CA 역할을하는 기본 '루트' CA 공급자가 있습니다. CA 프레임 워크는 호스트를 설정하는 동안 인증서 발급, 갱신, 해지 및 인증서 전파에 참여합니다. 이 프레임 워크는 Mold 관리 서버, KVM/LXC/SSVM/CPVM 에이전트 및 피어 관리 서버 간의 통신을 보호하는 데 기본적으로 사용됩니다.

다음은 이 기능의 다양한 측면을 제어하는 몇 가지 전역 설정입니다.

글로벌 설정	기술
ca.framework.provider.plugin	구성된 CA 공급자 플러그인
ca.framework.cert.keysize	인증서 생성에 사용되는 키 크기
ca.framework.cert.signature.algorithm	인증서 서명 알고리즘
ca.framework.cert.validity.period	인증서 유효 기간 (일)
ca.framework.cert.automatic.renewal	호스트에서 만료되는 인증서를 자동 갱신 할지 여부
ca.framework.background.task.delay	각 CA 백그라운드 작업 라운드 간의 지연 (초)
ca.framework.cert.expiry.alert.period	만료되는 인증서를 확인하고 경고하는 일 수
ca.plugin.root.private.key	(데이터베이스에 숨김 / 암호화 됨) 자동 생성된 CA 개인 키
ca.plugin.root.public.key	(데이터베이스에 숨김 / 암호화 됨) CA 공 개 키
ca.plugin.root.ca.certificate	(데이터베이스에 숨김 / 암호화 됨) CA 인 증서
ca.plugin.root.issuer.dn	루트 CA 공급자가 사용하는 CA 발급 고유 이름
ca.plugin.root.auth.strictness	양방향 SSL 인증 및 신뢰 유효성 검사를 시행하도록 설정
ca.plugin.root.allow.expired.cert	만료된 인증서가 있는 클라이언트를 허용 하도록 설정

`ca.framework.background.task.delay` 설정을 변경하려면 Mold 관리 서버가 시작될 때만 스레드 풀 및 백그라운드 작업이 구성되므로 관리 서버를 다시 시작해야 합니다.

CA 프레임 워크는 기본적으로 `root` CA 공급자를 사용합니다. 이 CA 공급자는 업그레이드 후 처음 부팅할 때 개인/공개 키와 CA 인증서를 자동으로 생성합니다. 새로 설치된 환경의 경우 `ca.plugin.root.auth.strictness` 설정은 클라이언트와 서버 구성 요소 간의 양방향 SSL 인증 및 신뢰 유효성 검사를 시행하기 위해 `true` 가 되지만 업그레이드된 환경에서는 기존과 역 호환 되도록 `false` 가 됩니다. 모든 클라이언트와 서버를 신뢰하는 동작과 단방향 SSL 인증.

업그레이드된 / 기존 환경은 `provisionCertificate` API를 사용하여 이미 연결된 에이전트/호스트에 대한 인증서를 갱신/설정할 수 있으며, 모든 에이전트/호스트가 보안되면 `ca.plugin.root.auth.strictness` 를 `true` 로 설정하여 인증 및 유효성 검사 엄격성을 적용할 수 있으며 관리 서버를 다시 시작합니다.

서버 주소 사용

여러 관리 서버를 사용하는 경우 tcp-LB는 관리 서버의 포트 8250 (기본값)에서 사용되며 VIP/LB-IP는 KVM과 같은 다양한 Mold 에이전트에서 사용할 `host` 설정으로 사용됩니다. CPVM, SSVM 에이전트, 포트 8250의 `host` 에 연결합니다. `host` 설정은 새 Mold 호스트/에이전트가 서플된 목록을 가져올 관리 서버 IP의 쉼표로 구분된 목록을 허용할 수 있습니다. 라운드 로빈 방식으로 재 연결을 순환할 수 있는 것과 동일합니다.

여러 관리 서버를 사용하는 경우 tcp-LB는 관리 서버의 포트 8250 (기본값)에서 사용되며 VIP/LB-IP는 `host` KVM, CPVM과 같은 다양한 Mold 에이전트에서 사용되는 설정으로 사용됩니다. `host` 포트 8250에 연결하는 SSVM 에이전트. `host` 설정은 새 Mold 호스트/에이전트가 재 연결을 순환할 수 있는 동일한 목록을 서플된 목록으로 가져 오는 쉼표로 구분된 관리 서버 IP 목록을 허용할 수 있습니다. 라운드 로빈 방식.

보안 프로세스

관리 서버에 연결/재 연결하는 동안 에이전트는 서버 인증서의 유효성을 검사하고 `ccloud.jks` 에 액세스할 수 있을 때 클라이언트 인증서 (발급 됨)를 제시할 수 있습니다. 이 기능 이전에 설정된 이전 호스트에서는 키 저장소를 사용할 수 없지만 `ca.plugin.root.auth.strictness` 가 `false` 로 설정된 경우 관리 서버에 계속 연결할 수 있습니다. 관리 서버는 시작시 자체 `ccloud.jks` 키 저장소를 확인하고 설정합니다. 이 키 저장소는 피어 관리 서버에 연결하는 데 사용됩니다.

KVM 호스트 추가 또는 systemvm 호스트 시작과 같은 새 호스트가 설정되면 CA 프레임 워크가 시작되고 ssh를 사용하여 `keystore-setup` 을 실행하여 새 키 저장소 파일 `ccloud.jks.new` 를 생성하고 저장합니다. 에이전트의 속성 파일 및 CSR `ccloud.csr` 파일에 있는 키 저장소의 임의 암호. 그런 다음 CSR을 사용하여 해당 에이전트/호스트에 대한 인증서를 발급하고 ssh를 사용하여 `keystore-cert-import` 를 실행하여 발급된 인증서를 CA 인증서와 함께 가져옵니다. 키 저장소는 `ccloud.jks` 로 이름이 변경되었습니다. 사용중인 이전 키 저장소를 대체합니다. 이 과정에서 키 및 인증서 파일은 에이전트의 구성 디렉터리에 있는 `ccloud.key` , `ccloud.crt` , `ccloud.ca.crt` 에도 저장됩니다.

호스트가 대역 외 추가 (예 : Mold 외부에서 먼저 설정되고 클러스터에 추가된 KVM 호스트)하면 키 저장소 파일을 사용할 수 없지만 키 저장소 유틸리티 스크립트를 수동으로 사용하여 키 저장소 및 보안을 설정할 수 있습니다. `keystore-setup` 을 먼저 실행하여 키 저장소 및 CSR을 생성한 다음 Mold CA를 사용하여 CSR을 `issueCertificate` API에 제공하여 인증서를 발급할 수 있으며, 마지막으로 발급된 인증서와 CA 인증서는 `keystore-cert-import` 스크립트를 사용하여 키 저장소로 가져 왔습니다.

다음은 이러한 스크립트의 사용법을 나열합니다. 이러한 스크립트를 사용할 때 전체 경로를 사용할 때 최종 키 저장소 파일 이름을 `ccloud.jks` 로 사용하고 인증서/키 콘텐츠를 인코딩하고 줄 바꿈이 `^` 로 대체되도록 제공해야 합니다. 공백은 `~` 로 대체됩니다. :

```
keystore-setup <properties file> <keystore file> <passphrase> <validity> <csr file>
keystore-cert-import <properties file> <keystore file> <mode: ssh|agent> <cert file> <cert content> <ca-cert file> <ca-cert content> <private-key file> <private key content:optional>
```

KVM 호스트는 에이전트 및 libvirt 프로세스 모두에 대한 키 저장소 및 인증서 설정이 있는 경우 보안된 것으로 간주됩니다. 보안 호스트는 TLS 지원 라이브 VM 마이그레이션 만 허용하고 시작합니다. 이를 위해서는 libvirt가 기본 포트 16514에서 수신 대기하고 포트가 방화벽 규칙에서 허용되어야 합니다. 인증서 갱신 (`provisionCertificate` API 사용)은 새 인증서를 배포한 후 libvirt 프로세스와 에이전트를 모두 다시 시작합니다.

KVM Libvirt 후크 스크립트 포함

기능 개요

- 이 기능은 KVM 호스트에 적용됩니다.
- Mold에서 사용되는 KVM은 Libvirt 문서 페이지 후크에 설명된대로 표준 Libvirt 후크 스크립트 동작을 사용합니다.
- KVM Mold 에이전트를 설치하는 동안 이후 qemu 스크립트라고하는 Libvirt 후크 스크립트 `"/etc/libvirt/hooks/qemu"`가 설치됩니다.
- 이것은 Libvirt 후크 사양에 따라 VM이 시작, 중지 또는 마이그레이션될 때마다 네트워크 관리 작업을 수행하는 Python 스크립트입니다.
- 사용자 정의 네트워크 구성 작업은 qemu 스크립트가 호출되는 동시에 수행될 수 있습니다.
- 문제의 작업은 사용자별로 다르기 때문에 Mold에서 제공하는 qemu 스크립트에 포함될 수 없습니다.
- Libvirt 문서 페이지 qemu 는 각 VM에서 KVM 및 Libvirt가 수행하는 작업을 기반으로 qemu 스크립트에 전달할 수 있는 매개 변수를 설명합니다. 'prepare', 'start', 'started', 'stopped', 'release', 'migrate', 'restore', 'reconnect' and 'attach'.

KVM Libvirt Hook 스크립트는 다음을 허용합니다.

1. 추가 네트워킹 구성 작업을 수행하기 위한 사용자 지정 스크립트의 포함 및 실행.
2. 포함된 사용자 지정 스크립트는 bash 스크립트 또는 python 스크립트 일 수 있습니다.
3. 각 사용자 지정 스크립트의 시작 및 반환 명령이 캡처되고 기록됩니다.
4. 포함하거나 호출할 수 있는 사용자 지정 스크립트의 수에는 제한이 없습니다.

용법

- cloudstack-agent 패키지는 Libvirt의 `/etc/libvirt/hooks` 디렉토리에 qemu 스크립트를 설치합니다.
- Libvirt 문서 페이지 인수 는 qemu 스크립트에 전달할 수 있는 인수를 설명합니다.
- 입력 인수는 다음과 같습니다.
 - a. 작업에 관련된 개체의 이름과 없는 경우 '-'. 예를 들어, 시작 중인 게스트의 이름입니다.
 - b. 수행 중인 작업의 이름입니다. 예를 들어 게스트가 시작되는 경우 '시작'입니다.
 - c. 하위 작업 표시와 없는 경우 '-'.
 - d. 추가 인수 문자열과 없는 경우 '-'.

- 작업 인수는 KVM 및 Libvirt가 각 VM에서 수행하는 작업 ('prepare', 'start', 'started', 'stopped', 'release', 'migrate', 'restore', 'reconnect', 'attach')을 기반으로 합니다.
- 유효하지 않은 작업 인수가 수신되면 qemu 스크립트는 사용자 정의 스크립트를 실행하지 않고 종료합니다.
- qemu 스크립트에 전달되는 모든 입력 인수는 각 사용자 정의 스크립트에도 전달됩니다.
- 위의 각 작업에 대해 qemu 스크립트는 사용자 정의 포함 경로 /etc/libvirt/hooks/custom/에서 `<action>_<custom script name>` 이름으로 스크립트를 찾아 실행합니다. 이 명명 규칙을 따르지 않는 사용자 지정 스크립트는 무시되고 실행되지 않습니다.
- Libvirt 표준 작업 외에도 qemu 스크립트는 스크립트 이름 앞에 "all"이 붙은 사용자 지정 스크립트를 찾아 실행합니다. 예: `all_<custom script name>`. 이러한 사용자 지정 스크립트는 유효한 Libvirt 작업으로 qemu 스크립트가 호출될 때마다 실행됩니다.
- 사용자 지정 스크립트가 여러 개인 경우 정렬된 (알파벳순) 순서로 실행됩니다. 알파벳 순서는 파일 이름에서 접두사와 밑줄을 제거한 후 파일 이름 부분을 사용합니다. 예를 들어, 디렉토리에 두 개의 사용자 정의 스크립트 파일이 있는 경우: `all_cde.sh`, `migrate_abc.py`; 다음 순서로 정렬되고 실행됩니다: `migrate_abc.py`, `all_cde.sh`.
- 커스텀 스크립트는 bash 스크립트 또는 python 스크립트 일 수 있습니다.
- 사용자 지정 스크립트는 기본 호스트 운영 체제에서 실행 가능해야 합니다. 실행 불가능한 스크립트는 기록되고 무시됩니다.
- 각 사용자 지정 스크립트의 시작 및 반환 명령이 캡처되고 기록됩니다.
- 사용자 지정 스크립트를 실행하는 동안 사용자 지정 스크립트의 표준 출력 (stdout) 및 표준 오류 (stderr) 출력이 `/var/log/libvirt/qemu-hook.log`에 기록 (추가)됩니다. 사용자 정의 스크립트가 어떤 것을 기록해야 하는 경우 이 파일을 로깅 목적으로 사용할 수도 있습니다.
- qemu 스크립트에는 사용자 정의 스크립트 실행이 시작될 때마다 카운트 다운하는 시간 초과 설정이 있습니다. 시간 초과 시간이 경과한 후에도 사용자 지정 스크립트가 계속 실행 중이면 사용자 지정 스크립트가 정상적으로 종료됩니다.

시간 초과 구성

- qemu 스크립트 상단에 있는 "timeoutSeconds"라는 시간 초과 설정은 10 분의 기본 시간 초과 설정 (10 * 60 초로 표시됨)을 가지며 다른 시간 초과가 필요한 경우 수동으로 수정할 수 있습니다.
- 다른 시간 제한을 구성하려면 다음을 수행하십시오.
 - a. /etc/libvirt/hooks/ 폴더로 이동합니다.
 - b. 편집기에서 qemu 스크립트를 엽니다.
 - c. "timeoutSeconds" 시간 제한 설정을 찾습니다.
 - d. 10 * 60 값을 원하는 제한 시간 값으로 변경하십시오. 예를 들어 20 분 제한 시간의 경우 20 * 60입니다.

특정 VM 작업에 대한 사용자 지정 스크립트 이름 지정

- 특정 VM 작업이 끝날 때 실행해야 하는 사용자 지정 스크립트의 경우 다음을 수행합니다.
 - a. 특정 작업에 대해 실행해야 하는 사용자 지정 스크립트로 이동합니다.
 - b. 파일 이름 앞에 특정 작업 이름과 밑줄을 붙여 파일 이름을 바꿉니다. 예를 들어 사용자 정의 스크립트의 이름이 `abc.sh` 인 경우 이름 앞에 'migrate'와 밑줄을 붙여 `migrate_abc.sh`가 됩니다.

모든 VM 작업에 대한 사용자 지정 스크립트 이름 지정

- 모든 VM 작업이 끝날 때 실행해야 하는 사용자 지정 스크립트의 경우 다음을 수행합니다.
 - a. 모든 작업에 대해 실행해야 하는 사용자 지정 스크립트로 이동합니다.
 - b. 파일 이름에 'all' 접두사를 붙이고 뒤에 밑줄을 붙여 파일 이름을 바꿉니다. 예를 들어 사용자 지정 스크립트의 이름이 def.py인 경우 이름 앞에 'all'과 밑줄을 붙여 all_def.py가 됩니다.

사용자 지정 스크립트 실행 구성

- 기본 호스트 운영 체제에서 실행할 수 있도록 각 사용자 지정 스크립트 실행 권한을 부여합니다.
 - a. 실행 가능해야 하는 사용자 지정 스크립트로 이동합니다.
 - b. 사용자 지정 스크립트 실행 권한을 부여합니다.
- qemu 스크립트가 찾아서 실행할 수 있도록 사용자 정의 포함 폴더/etc/libvirt/hooks/ custom /에 사용자 정의 스크립트를 배치하십시오.
 - a. /etc/libvirt/hooks/custom/ 폴더가 생성되었고 올바른 액세스 권한이 있는지 확인하십시오.
 - b. 복사해야 하는 사용자 지정 스크립트로 이동합니다.
 - c. 스크립트를/etc/libvirt/hooks/custom/ 폴더에 복사합니다.
- 셸에서 사용자 정의 스크립트는 파일의 첫 번째 줄에 #! /bin/bash를 포함하여 스크립트가 bash로 실행되도록 합니다.
- Python에서 사용자 지정 스크립트는 파일의 첫 번째 줄에 #! /usr/bin/python을 포함하여 스크립트가 python으로 실행되도록 합니다.

KVM 롤링 유지 관리

개요

Mold는 사용자 지정 스크립트를 실행하여 Zone, Pod 또는 클러스터 내에서 KVM 호스트의 업그레이드 또는 패치 프로세스를 자동화하기 위한 유연한 프레임 워크를 제공합니다. 이러한 스크립트는 스테이지 컨텍스트에서 실행됩니다. 각 단계는 실행할 사용자 지정 스크립트를 하나만 정의합니다.

KVM 롤링 유지 관리 프로세스에는 4 단계가 있습니다.

1. Pre-Flight 단계 : Pre-flight 스크립트 (`PreFlight` 또는 `PreFlight.sh` 또는 `PreFlight.py`)는 롤링 유지 관리를 시작하기 전에 호스트에서 실행됩니다. 실행 전 확인 스크립트가 호스트에서 오류를 반환하면 수행된 작업없이 롤링 유지 관리가 취소되고 오류가 반환됩니다. 사전 실행 스크립트가 정의되지 않은 경우 호스트에서 확인이 수행되지 않습니다.
2. 유지 관리 전 단계 : 특정 호스트를 유지 관리하기 전에 유지 관리 전 스크립트 (`PreMaintenance` 또는 `PreMaintenance.sh` 또는 `PreMaintenance.py`)가 실행됩니다. 사전 유지 관리 스크립트가 정의되어 있지 않으면 사전 유지 관리 작업이 수행되지 않으며 관리 서버는 호스트를 유지 관리로 바로 이동한 다음 에이전트가 유지 관리 스크립트를 실행하도록 요청합니다.
3. 유지 관리 단계 : 유지 관리 스크립트 (`Maintenance` 또는 `Maintenance.sh` 또는 `Maintenance.py`)는 호스트가 유지 관리에 들어간 후에 실행됩니다. 유지 관리 스크립트가 정의되지 않았거나 사전 또는 사전 유지 관리 스크립트가 유지 관리가 필요하지 않다고 결정하는 경우 호스트는 유지 관리에 들어 가지 않으며 사전 유지 관리 스크립트가 완료되면 모든 작업이 끝났음을 알립니다. 유지 관리 작업과 KVM 에이전트는 호스트를 관리 서버로 다시 넘깁니다. 유지 관리 스크립트가 완료되

었다는 신호를 보내면 Host Agent는 유지 관리 작업이 완료되었음을 관리 서버에 알립니다. 따라서 호스트는 유지 관리 모드와 수집된 모든 '정보'(예: 처리 시간)를 종료할 준비가 된 것입니다.) 이 관리 서버로 반환됩니다.

4. 유지 관리 후 단계 : 유지 관리 후 스크립트 ((`PostMaintenance` 또는 `PostMaintenance.sh` 또는 `PostMaintenance.py`))는 호스트가 유지 관리를 종료한 후 유효성 검사를 수행할 것으로 예상됩니다. 이러한 스크립트는 재부팅 또는 스크립트 내 재시작을 포함하여 유지 관리 프로세스 중에 문제를 감지하는 데 도움이 됩니다.

Note

실행 전 및 유지 관리 전 스크립트의 실행을 통해 호스트에 유지 관리 단계가 필요하지 않은지 확인할 수 있습니다. 비행 전 또는 유지 관리 전 스크립트의 특수 종료 코드 = 70은 호스트에 유지 관리 단계가 필요하지 않음을 Mold에 알려줍니다.

관리자는 단계당 하나의 스크립트만 정의해야 합니다. 단계에 스크립트가 포함되지 않은 경우 건너뛰고 다음 단계를 계속합니다. 관리자는 스크립트를 정의하고 호스트에 복사할 책임이 있습니다.

Note

관리자는 모든 KVM 호스트에서 스크립트의 유지 관리 및 복사를 담당합니다.

롤링 유지 관리를 수행할 모든 KVM 호스트에는 두 가지 유형의 스크립트 실행 접근 방식이 있습니다.

- Systemd 서비스 실행자 : 이 접근 방식은 systemd 서비스를 사용하여 스크립트 실행을 호출합니다. 스크립트가 실행을 마치면 파일에 내용을 쓰고 에이전트가 결과를 읽고 관리 서버로 다시 보냅니다.
- 에이전트 실행자 : Mold 에이전트는 JVM 내에서 스크립트 실행을 호출합니다. 에이전트가 중지되거나 다시 시작되는 경우 관리 서버는 에이전트가 다시 연결될 때 단계가 완료된 것으로 간주합니다. 이 접근 방식은 상태를 파일에 보관하지 않습니다.

구성

롤링 유지 관리 프로세스는 관리 서버에서 다음 전역 설정을 통해 구성할 수 있습니다.

- `kvm.rolling.maintenance.stage.timeout` : 호스트에서 관리 서버로의 롤링 유지 관리 단계 업데이트에 대한 제한 시간 (초)을 정의합니다. 기본값은 1800입니다. 이 제한 시간은 단계별로 관찰됩니다.
- `kvm.rolling.maintenance.ping.interval` : 롤링 유지 관리 중 단계를 수행하는 관리 서버와 호스트 간의 ping 간격 (초)을 정의합니다. 관리 서버는 'ping 간격'초마다 호스트의 업데이트를 확인합니다. 기본값은 10입니다.
- `kvm.rolling.maintenance.wait.maintenance.timeout` : 롤링 유지 관리 프로세스의 일부로 유지 관리 모드로 들어가고 준비하는 호스트를 기다리는 시간 제한 (초)을 정의합니다. 기본값은 1800입니다.

각 KVM 호스트에서 관리자는 스크립트가 정의된 디렉토리를 표시하고 `agent.properties` 파일을 편집 하고 속성을 추가해야 합니다.

- `rolling.maintenance.hooks.dir=<SCRIPTS_DIR>`

선택적으로 관리자는 각 호스트의 롤링 유지 관리 스크립트에 대해 systemd 실행기를 비활성화 (기본적으로 활성화 됨)하여 에이전트가 에이전트 실행을 통해 스크립트를 호출할 수 있도록 할 수 있습니다. 이는 `agent.properties` 파일을 편집 하고 속성을 추가하여 수행할 수 있습니다.

- `rolling.maintenance.service.executor.disabled=true`

용법

관리자는 `startRollingMaintenance` 하나 이상의 Zone, Pod, 클러스터 또는 호스트를 선택 하여 API 또는 UI를 통해 롤링 유지 관리 프로세스를 호출할 수 있습니다.

`startRollingMaintenanceAPI` 는 다음과 같은 매개 변수를 사용할 수 :

- `hostids` , `clusterids` , `podids` 및 `zoneids` 상호 배타적, 오직 그 중 하나가 전달되어야한다. 언급된 각 매개 변수에는 정의하는 엔티티의 ID 목록이 쉼표로 구분되어 있어야 합니다.
- `forced` : 선택적 부울 매개 변수, 기본값은 false. 활성화된 경우 롤링 유지 관리 프로세스에서 오류가 발생하는 경우 호스트 반복을 중지하지 않습니다.
- `timeout` : 선택적 매개 변수로 호스트에서 단계가 완료되는 시간 제한 (초)을 정의합니다. 이 매개 변수는 전역 설정에 정의된 시간 초과보다 우선합니다 `kvm.rolling.maintenance.stage.timeout`.

Note

제한 시간 (API 매개 변수 또는 글로벌 설정에 의해 정의 됨)은 글로벌 설정 'kvm.rolling.maintenance.ping.interval'에 정의된 핑 간격보다 크거나 같아야합니다. 시간 초과가 핑 간격보다 낮은 경우 API는 유지 관리 작업을 시작하지 않고 설명 메시지와 함께 빠르게 실패합니다.

- `payload` : 선택적 문자열 매개 변수, 각 단계의 스크립트에 전달할 추가 인수를 추가합니다. 매개 변수로 설정된 문자열은 스크립트 호출 끝에 페이로드를 추가하여 각 단계의 롤링 유지 관리 프로세스와 관련된 각 스크립트를 호출하는 데 사용됩니다.

Note

페이로드 매개 변수는 각 단계 스크립트 실행의 끝에 추가됩니다. 이를 통해 관리자는 매개 변수를 수락하고 페이로드 매개 변수를 통해 각 단계 실행에 전달할 수 있는 스크립트를 정의할 수 있습니다. 예를 들어, 페이로드 매개 변수를 "param1=val1 param2=val2"로 정의하면 `execute: './script param1=val1 param2=val2'`와 유사하게 두 매개 변수를 각 단계 실행에 전달합니다.

UI에서 관리자는 하나 이상의 Zone, Pod, 클러스터 또는 호스트를 선택하고 롤링 유지관리 시작 버튼을 클릭해야 합니다.

Note

롤링 유지 관리 작업 결과는 UI에 표시되지 않습니다. 작업 출력을 보려면 API/CLI (예 : CloudMonkey)를 사용해야 합니다.

방법

유지 관리 작업을 시도하기 전에 모든 호스트에서 실행 전 및 용량 확인이 수행됩니다.

1. 관리 서버는 지정된 범위의 모든 호스트가 유지 관리로 설정될 수 있도록 용량 검사를 수행합니다. 이러한 검사에는 호스트 태그, 선호도 그룹 및 계산 검사가 포함됩니다.
2. 사전 실행 스크립트는 각 호스트에서 실행됩니다. 이러한 스크립트 중 하나라도 실패하면 'force' 매개 변수를 사용하지 않는 한 아무 작업도 수행되지 않습니다.

비행 전 스크립트는 호스트에서 유지 관리가 필요하지 않다는 신호를 보낼 수 있습니다. 이 경우 호스트는 롤링 유지 관리 호스트 반복에서 건너뛰니다.

사전 검사가 통과되면 관리 서버는 선택한 범위의 각 호스트를 반복하고 나머지 단계를 순서대로 실행하는 명령을 보냅니다. 선택한 범위의 호스트는 클러스터별로 그룹화되므로 다른 클러스터의 호스트를 처리하기 전에 클러스터의 모든 호스트가 처리됩니다.

관리 서버는 선택한 범위에 있는 각 클러스터의 호스트를 반복하고 각 호스트에 대해 다음을 수행합니다.

- 클러스터를 비활성화합니다 (이전에 비활성화되지 않은 경우).
- 호스트에 유지 관리 스크립트가 있는지 확인합니다 (이 검사는 나머지 단계가 아닌 유지 관리 스크립트에 대해서만 수행됨).
 - 호스트에 유지 관리 스크립트가 포함되지 않은 경우 호스트를 건너뛰고 클러스터의 다음 호스트에서 반복이 계속됩니다.
- 유지 관리 모드로 들어가기 전에 유지 관리 전 스크립트 (있는 경우)를 실행합니다.
 - 유지 관리 전 스크립트는 호스트에서 유지 관리가 필요하지 않다는 신호를 보낼 수 있습니다. 이 경우 호스트를 건너뛰고 클러스터의 다음 호스트에서 반복이 계속됩니다.
 - 유지 관리 전 스크립트가 실패하고 'forced' 매개 변수가 설정되지 않은 경우 롤링 유지 관리 프로세스가 실패하고 오류가 보고됩니다. 'forced' 매개 변수가 설정되면 호스트를 건너뛰고 클러스터의 다음 호스트에서 반복이 계속됩니다.
- 호스트가 유지 관리 모드로 들어갈 수 있는지 확인하기 위해 용량 검사가 다시 계산됩니다.

Note

용량을 다시 계산하기 전에 'fetchLatest' 매개 변수를 true로 설정하여 listCapacity API 실행을 수행하는 것과 유사하게 용량이 업데이트됩니다.

- 호스트는 유지 관리 모드로 들어가도록 지시합니다. 호스트가 'kvm.rolling.maintenance.wait.maintenance.timeout' 초 후에 유지 관리 모드로 전환되지 않으면 예외가 발생하고 API 실행이 중지되지만, 호스트는 결국 유지 관리 모드에 도달할 수 있습니다. 롤링 유지 보수 API / 코드의 제어.
- 호스트가 유지 관리되는 동안 유지 관리 스크립트 (있는 경우)를 실행합니다.
 - 유지 관리 스크립트가 실패하고 'forced' 매개 변수가 설정되지 않은 경우 롤링 유지 관리 프로세스가 실패하고 유지 관리 모드가 취소되고 오류가 보고됩니다. 'forced' 매개 변수가 설정되면 호스트를 건너뛰고 클러스터의 다음 호스트에서 반복이 계속됩니다.
- 유지 관리 모드 취소
- 유지 관리 모드를 취소한 후 유지 관리 후 스크립트 (있는 경우)를 실행합니다.
 - 유지 관리 후 스크립트가 실패하고 'forced' 매개 변수가 설정되지 않은 경우 롤링 유지 관리 프로세스가 실패하고 오류가 보고됩니다. 'forced' 매개 변수가 설정되면 호스트를 건너뛰고 클러스터의 다음 호스트에서 반복이 계속됩니다.
- 클러스터의 모든 호스트가 처리된 후 또는 오류가 발생한 경우 비활성화된 클러스터를 활성화합니다.

ABLESTACK Online Docs